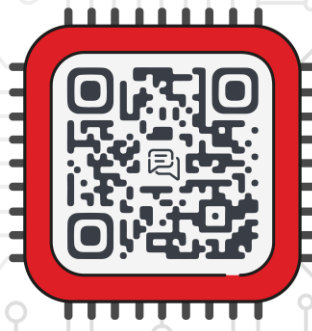




3

15

2250 sp

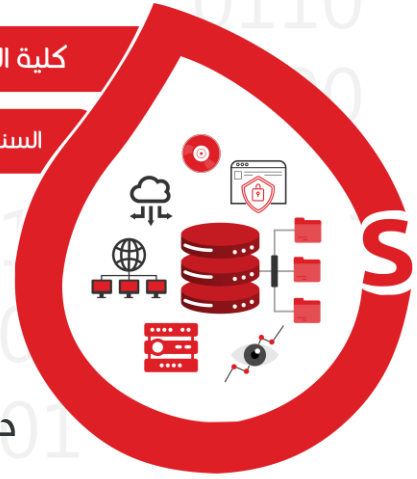
نظري

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Storage and File Structure - Part2

د. مادلين عبود



RB Informatics; 23/10/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

في المحاضرة السابقة (Part 1) وضعنا الأساس لفهم كيفية تعامل أنظمة قواعد البيانات مع التخزين الفيزيائي.

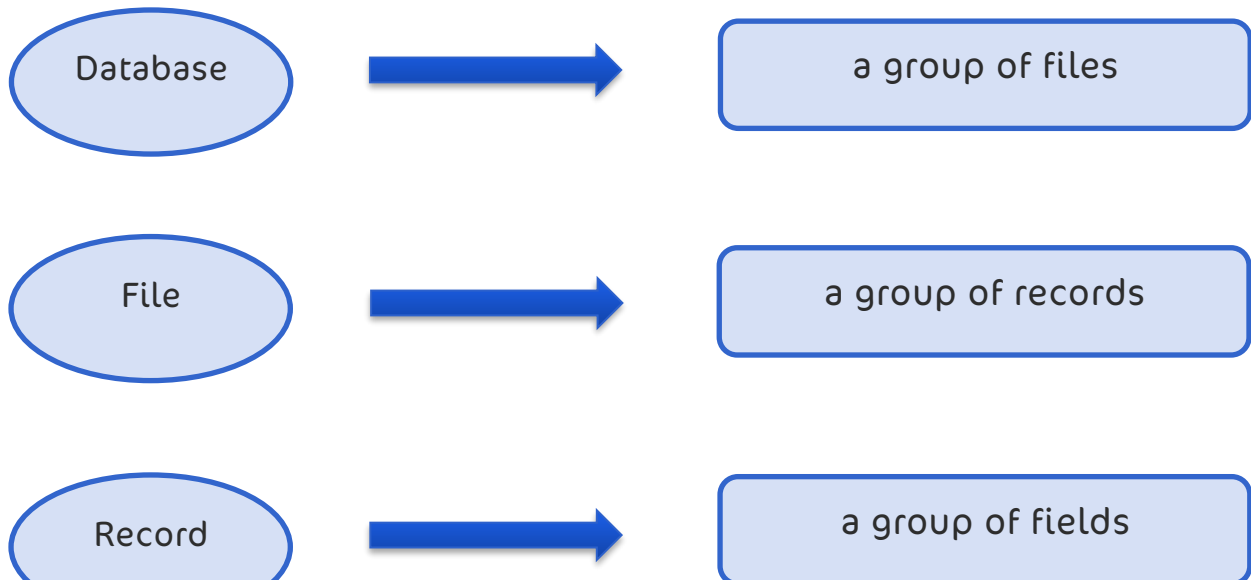
تعرفنا على وسائل التخزين بأنواعها، هرمية التخزين، وآليات تحسين أداء الأقراص مثل Buffering وال RAID، بالإضافة إلى فهم كيفية تنظيم البيانات على القرص من خلال المسارات والقطاعات والأسطوانات.

هذا الفهم مهم لأنه يمثل الطبقة الأدنى التي تعتمد عليها كل أنظمة تخزين الملفات داخل قواعد البيانات.

في هذا الجزء (Part 2) سنرتقي خطوة للأعلى، من التخزين الفيزيائي إلى تنظيم الملفات والسجلات، لنفهم كيف تُخزن العلاقات (الجدول) فعلياً داخل ملفات النظام، وكيف تُرتب السجلات داخل الكتل، وكيف تُدار عمليات الإدراج، الحذف، التعديل، وإعادة الاستخدام.

تنظيم الملفات

تخزن قاعدة البيانات كمجموعة من الملفات، حيث يتكون كل ملف من مجموعة من التسجيلات (صفوف)، وكل تسجيلية تتكون من مجموعة من الحقول (أعمدة).



السجلات ثابتة الطول (Fixed-Length Records)

هذه أبسط طريقة لتخزين الملفات، وتفترض:

- كل سجل له نفس الحجم تماماً (record size is fixed).
- كل ملف يحتوي سجلات من نفس النوع فقط (each file has records of one particular type only).

مثلا ملف خاص بجدول الطلاب

وملف آخر خاص بجدول المقررات... وهكذا

- كل علاقة (جدول) تُخزن في ملف مستقل (different files are used for different relations).

لماذا تعتبر هذه الطريقة بسيطة؟

لأننا نستطيع حساب مكان أي سجل بشكل مباشر.

كيف نحدد أين يوجد السجل رقم i؟

بما أن كل سجل حجمه ثابت n ثابت:

- يبدأ السجل رقم 1 عند البايت 0.
- يبدأ السجل رقم 2 عند البايت n.
- السجل رقم i يبدأ عند: $n \times (i - 1)$

وهذا يجعل البحث عن سجل معين سريع جداً، لأنك لا تحتاج للمرور على من قبله.

ولكن: يجب عدم السماح بتخزين تسجيلية في كتلتين.

المشكلة: السجلات تعبر حدود الكتل.

الملفات لا تقرأ بايتاً بايتاً، بل تقرأ على شكل كتل (Blocks)، كل كتلة لها حجم معين مثل 4KB.

لو كان السجل يبدأ في نهاية كتلة ولا يوجد مكان كافٍ ليكتمل داخل نفس الكتلة، فإنه سيتمدد إلى الكتلة التالية. وهذا يجعل قراءة السجل تحتاج كتلتين بدل واحدة مما يؤدي إلى أداء أقل.

الحل: منع السجل من عبور حدود الكتلة.

إذا لم يتبق مكان في الكتلة، نترك فراغاً بسيطاً وننتقل للكتلة التالية بالكامل لنضع السجل الجديد.

توضيح للفكرة:

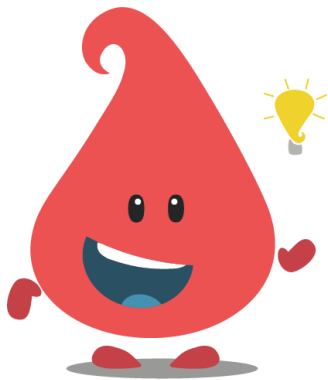
ما المشكلة؟

افترض أن لدينا تسجيلية طولها 300 بايت.

وفي نهاية كتلة معينة، لم يتبق سوى 100 بايت فارغة.

ماذا يحدث لو قمنا بتخزين السجل هنا؟

- أول 100 بايت من السجل تُخزن في نهاية الكتلة الحالية.
- الـ 200 بايت المتبقية تُخزن في الكتلة التالية.



بهذا يصبح السجل موزعاً هكذا:

كتلة رقم 7: آخر 100 بايت من السجل.

كتلة رقم 8: أول 200 بايت من السجل.

لماذا هذا خطر وغير مرغوب؟

لأن:

1. قراءة السجل ستحتاج لقراءة كتلتين بدلاً من كتلة واحدة

وهذا يزيد ال I/O والوقت.

2. تعقيد في إدارة الملفات

يجب حفظ معلومات إضافية عن كيفية متابعة السجل في الكتلة التالية.

3. زيادة احتمال الفساد (corruption) عند حصول أخطاء

إذا حُذفت كتلة أو تعطلت، السجل كله يتلف.

الحل: عدم السماح للسجل بتجاوز حدود الكتلة.

بمعنى:

إذا لم يتبق مكان كافٍ في الكتلة لتخزين السجل كاملاً..

لا نقوم بتخزينه هنا، بل ننتقل للكتلة التالية.

ولا نضع أي جزء من السجل في الكتلة السابقة.

كيف يتم تنفيذ ذلك؟

إذا كنا في كتلة حجمها 4096 بايت:

• متبقي فيها 100 بايت فقط.

• والسجل حجمه 300 بايت.

نقوم ب:

• ترك ال 100 بايت فارغةً تسمى (fragmentation)

• الانتقال للكتلة التالية، وتخزين السجل كاملاً فيها

كتلة رقم 7: 100 بايت فارغة.

كتلة رقم 8: 300 بايت (السجل كامل بها)

لماذا يعتبر هذا الأفضل؟

• سرعة أكبر: كل سجل محفوظ داخل كتلة واحدة فقط.

• تبسيط كبير في الكود.

• لا حاجة لتعقب سجل موزع عبر كتلتين.

• تقليل احتمالية الأخطاء.

الآن عندما قررنا:

- ألا نعبر بالسجلات حدود الكتلة.
 - وأن نضع كل سجل في مكان واحد فقط داخل كتلة واحدة.
- هذا يعني أن مساحة التخزين ستصبح فيها:
- مناطق ممتلئة بسجلات.
 - وأحياناً فراغات داخل الكتلة (لأن السجل لا يجد مكاناً مناسباً).

وهنا تأتي مشكلة مهمة:

ماذا نفعل بالمساحة التي أصبحت فارغة بعد حذف سجل؟
هل نتركها؟
هل نعيد ترتيب السجلات؟
أم نستخدمها لاحقاً؟

🔗 **وهنا تظهر ثلاث استراتيجيات للحذف:**

1. نقل كل السجلات بعده خطوة للأمام:

- إذا حذفنا السجل 5، ننقل السجلات 6 إلى 5، 7 إلى 6... وهكذا.
- الملف يبقى بدون فراغات.
- لكن العملية بطيئة جداً في الملفات الكبيرة.

2. نقل السجل الأخير إلى مكان السجل المحذوف:

- سريع جداً (نقل سجل واحد فقط).
- لكنه يغيّر ترتيب السجلات.

3. عدم نقل شيء، فقط نضع علامة محذوف:

- نترك السجل مكانه، لكنه يصبح سجلاً فارغاً يمكن إعادة استخدامه.
- نربطه بقائمة تسمى Free List.

وهذه الطريقة الأكثر استخداماً.

الحياة تعلّمك أن الخسائر جزء من الطريق

وليست نهاية الرحلة



ليكن لدينا العلاقة (account (account-num, branch-name, balance):

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 2	A-215	Mianus	700
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600
record 8	A-218	Perryridge	700

عند حذف record 2 بالطريقة 1 (تنقل كل السجلات التالية له خطوة):

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600
record 8	A-218	Perryridge	700

أما عند حذفه بالطريقة 2 (نقل السجل الأخير إلى مكان الحذف):

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 8	A-218	Perryridge	700
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600

فكرة ال Free List (إعادة استخدام السجلات المحذوفة):

- في رأس الملف file header، نخزن عنوان أول سجل محذوف.
 - داخل هذا السجل المحذوف، نخزن عنوان السجل المحذوف التالي.
 - وهكذا تصبح لدينا سلسلة من السجلات الفارغة المتاحة للإضافة.
- هذه العناوين تعمل مثل Pointers لأنها تشير إلى مكان السجل.

ميزة رائعة:

لا نحتاج مساحة إضافية، السجل المحذوف نفسه يُستخدم لتخزين العنوان.

header				
record 0	A-102	Perryridge	400	
record 1				
record 2	A-215	Mianus	700	
record 3	A-101	Downtown	500	
record 4				
record 5	A-201	Perryridge	900	
record 6				
record 7	A-110	Downtown	600	
record 8	A-218	Perryridge	700	

السجلات متغيرة الطول (Variable-Length Records)

لماذا نحتاج سجلات متغيرة الطول؟

في الطريقة الأولى (Fixed-Length Records) كان طول السجل ثابتاً، وهذا يعمل جيداً عندما:

- كل السجلات متشابهة.
- كل الحقول لها حجم ثابت.

لكن في الحقيقة... قواعد البيانات غالباً معقدة، وتظهر حالات لا يمكن أن تتبع هذا الأسلوب.

مثال 1: (ملفات تحتوي أكثر من نوع سجلات)

ملف واحد قد يحتوي:

1. سجل طالب
 2. سجل دكتور
 3. سجل مقرر
- كل نوع له عدد حقول مختلف وطول مختلف.

مثال 2: (حقول بطول غير ثابت)

1. نص طويل (text)
 2. وصف كتاب
 3. رسالة بريد إلكتروني
 4. عنوان (Address)
 5. قائمة أرقام هواتف
- هذه الحقول لا يمكن تحديد حجمها مسبقاً.

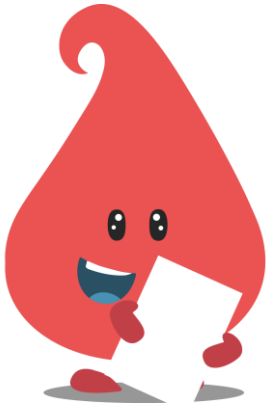
مثال 3: (سجل يحتوي قيم متعددة (multivalued))

1. طالب لديه أكثر من رقم هاتف
 2. كتاب له أكثر من مؤلف
 3. منتج له أكثر من صورة
- طول السجل سيختلف حسب عدد العناصر.

لماذا لا نستطيع استخدام الطريقة السابقة (Fixed-Length)؟

لأنها تعتمد على:

- نفس حجم السجل لجميع السجلات.
- حساب موقع السجل بالصيغة: $\text{byte_offset} = (i - 1) \times n$



لكن هنا:

- السجلات لا تتساوى في الطول.
- السجل 1 قد يكون 50 بايت.
- السجل 2 قد يكون 220 بايت.
- السجل 3 قد يكون 90 بايت.

إذن لا يمكن حساب مواقعهم بهذه السهولة.

إحدى الحلول البدائية لهذه المشكلة:

استخدام علامة (حرف خاص) نهاية السجل (Byte-String Representation of Variable-Length Records).
نخزن سجلاً كسلسلة من البايتات، ونستخدم رمزاً خاصاً مثل \n أو # لكي يدل على نهاية السجل، ورمزاً آخر لنهاية الحقل داخل السجل.

مثل:

Ahmed#25#Cairo@
Mohamed#30#Giza@

دلالة الرموز:

end of record = ⊥

@ = نهاية السجل

= نهاية الحقل

0	Perryridge	A-102	400	A-201	900	A-218	700	⊥
1	Round Hill	A-305	350	⊥				
2	Mianus	A-215	700	⊥				
3	Downtown	A-101	500	A-110	600	⊥		
4	Redwood	A-222	700	⊥				
5	Brighton	A-217	750	⊥				

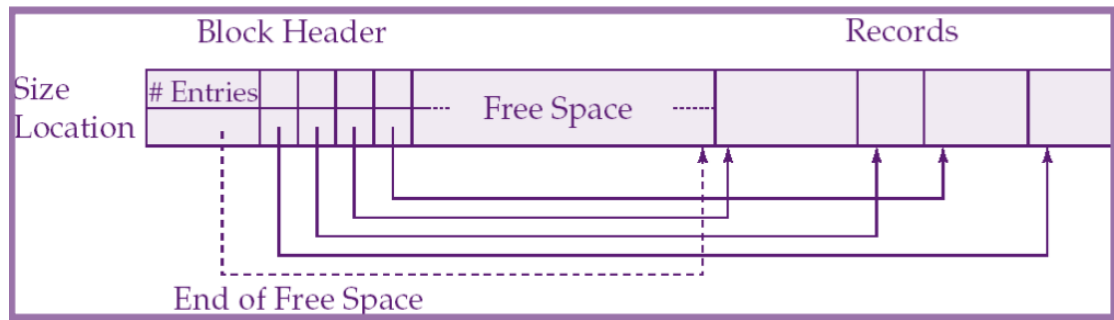
لكن هذه الطريقة غير مناسبة لقواعد البيانات الحقيقية والسبب:

1. الحذف صعب:
عندما تُحذف سجلاً من المنتصف، سيبقى هناك فراغ يحتاج معالجة.
2. التعديل أصعب:
لو كان السجل الجديد أطول من القديم، يجب نقل بقية السجلات.
3. الوصول العشوائي صعب:
لا يمكنك الوصول إلى السجل رقم أمباشرة، بل يجب المرور على كل ما قبله.
4. صعوبة إدارة الكتل (blocks):
قد يعبر السجل حدود الكتل لأن طوله غير معروف.
هذه مشاكل كبيرة.

بعض اللحظات تشفى بقطعة سعادة صغيرة



الحل الحقيقي المستخدم في قواعد البيانات Slotted Page (الصفحة المجزأة)



هي صفحة (Block) داخل نظام التخزين، تخصص لتخزين سجلات متغيرة الطول.
كل صفحة تحتوي:

- منطقة بيانات (Data Area).
- ترويسة (Header) تنظم كل شيء.

ترويسة (Header):

يوجد فيها:

1. عدد السجلات:

عدد سجلات هذه الصفحة الآن (الصفحة الحالية).

2. مؤشر نهاية المساحة الحرة (End of free space):

هذا يخبرنا:

- من أين تبدأ منطقة البيانات؟
- وأين يمكن وضع السجل القادم؟

3. جدول الفتحات (Slot Directory):

هذا أهم جزء:

لكل سجل يوجد فتحة (Slot Entry) تحتوي:

- إزاحة (Offset): بداية السجل داخل ال Data Area.
- الطول (Length): حجم السجل.

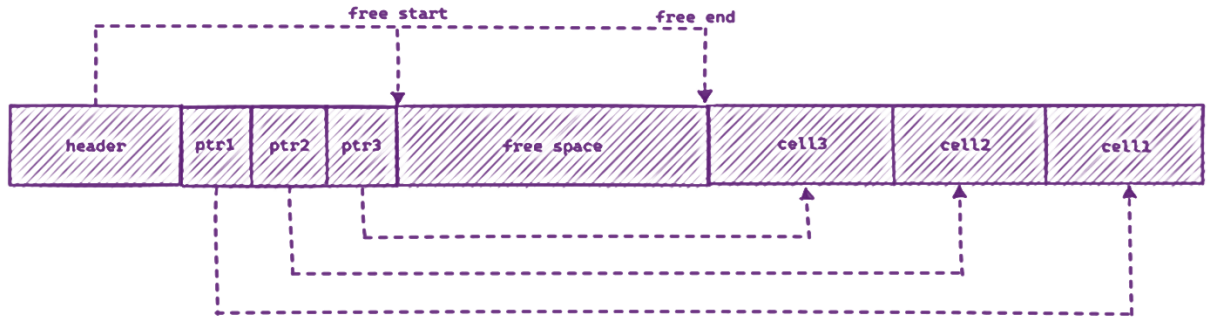
هذا الجدول هو سبب تسمية الصفحة ب Slotted Page.

القوة الحقيقية أن تخصص بعد كل سقوط

وكأنك لم تهزم يوماً

← منطقة البيانات Data Area:

هي المساحة التي تخزن فيها السجلات نفسها.



إذا باختصار:

- تحتوي ترويسة الصفحة (او الكتلة) على عدد التسجيلات المدخل إليها، نهاية المكان الفارغ في الكتلة، مكان توضع كل تسجيلية وحجمها.
- يمكن إزاحة التسجيلات داخل الصفحة بحيث نمنع وجود فراغات فيما بينها وبالتالي يجري تعديل ترويسة الصفحة.
- المؤشرات لا تؤثر مباشرة الى التسجيلات بل تؤثر الى مكان توضع عنوان التسجيلية في ترويسة الصفحة (أي تشير الى ال slot الذي بدوره يشير الى عنوان السجل).

السجلات متغيرة الطول تسبب مشكلتين كبيرتين:

1. ظهور فراغات داخل الصفحة بعد الحذف.
 2. الحاجة إلى نقل السجلات عند التعديل إذا تغير الطول.
- وهذا يجعل إدارة الملف أصعب، لذلك تقوم أنظمة قواعد البيانات بأحد الحلين:

1. تحويل السجلات إلى ثابتة الطول (Fixed-Length):

يتم إعطاء كل حقل أكبر حجم يمكن أن يصل إليه (الطول الأعظمي للتسجيلية). ويوضع في الحقول غير المستخدمة علامة عدم التعيين null أو علامة نهاية السجل:

0	Perryridge	A-102	400	A-201	900	A-218	700
1	Round Hill	A-305	350	⊥	⊥	⊥	⊥
2	Mianus	A-215	700	⊥	⊥	⊥	⊥
3	Downtown	A-101	500	A-110	600	⊥	⊥
4	Redwood	A-222	700	⊥	⊥	⊥	⊥
5	Brighton	A-217	750	⊥	⊥	⊥	⊥

مثلاً إذا كان الحقل اسم مستخدم، وكان الحد الأقصى 100 حرف، فقاعدة البيانات تحجز دائماً 100 حرف لكل سجل.

**الميزة:**

- لا يوجد مشاكل حذف/تعديل.
- لا يوجد تقطع داخل الصفحة.
- كل شيء سهل الانتقال والتنظيم.

العيب:

- تضحية كبيرة بالمساحة (size waste).

2. استخدام المؤشرات (Indirect Pointers):

السجل يبقى صغيراً، لكن الحقول الكبيرة (مثل النصوص والصور) يتم تخزينها:

- في صفحات أخرى.
- ويشير إليها السجل عبر Pointer.

0	Perryridge	A-102	400	
1	Round Hill	A-305	350	
2	Mianus	A-215	700	
3	Downtown	A-101	500	
4	Redwood	A-222	700	
5		A-201	900	
6	Brighton	A-217	750	
7		A-110	600	
8		A-218	700	

- يجري تمثيل التسجيلات متغيرة الطول بقائمة من التسجيلات ثابتة الحجم مربوطة مع بعضها بواسطة مؤشرات.
- يمكن استخدام هذه الطريقة حتى في حالة كان الطول الأعظمي للتسجيلة غير معلوم.

الميزة:

- التخلص من مشكلة التقطع داخل الصفحة.
- الحفاظ على مرونة الطول.

العيب:

- زيادة عمليات القراءة (I/O).
- ضياع في التخزين حيث أننا نحتاج لحجز مكان تخزين غير مستخدم.

لا ينتصر المجتهد بسرعة

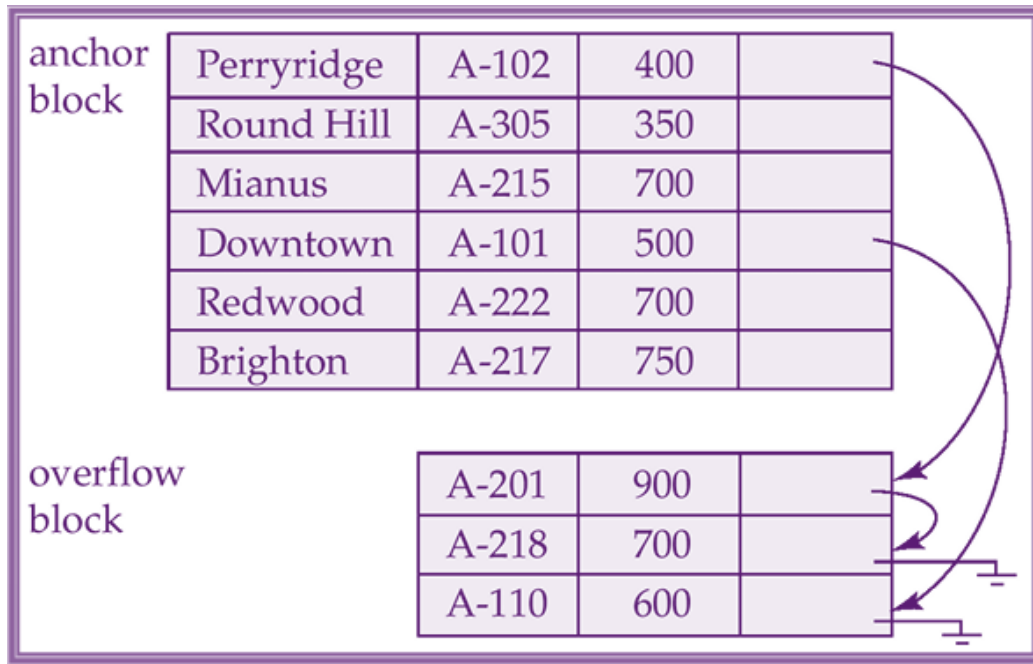
لكنه لا يهزم بسهولة



حل هذه المشكلة باستخدام نوعين من الكتل في الملف:

1. الكتلة المرجعية المعتمدة (Anchor Block):

- تحتوي السجلات الرئيسية.
 - معلومات أساسية + مؤشرات للبيانات الإضافية.
2. الكتلة الزائدة المضافة (Overflow Block):
- تحتوي البيانات الإضافية.
 - مرتبطة بالسجلات الرئيسية عبر المؤشرات.



تنظيم السجلات داخل الملفات

(File Organization Methods)

عندما تخزن قاعدة البيانات السجلات داخل الملفات، يجب أن تختار أسلوب التنظيم.

كل أسلوب له ميزة وعيب، ويؤثر على:

- سرعة البحث.
- سرعة الإضافة.
- مساحات التخزين.
- عدد عمليات الإدخال والإخراج (I/O).



العقل الهادئ يصنع قرارات عظيمة

لا تصنعها العجلة

هناك أربعة أساليب أساسية:

1. التخزين العشوائي (Heap File):

- السجل يُخزن في أي مكان فيه مساحة.
- بسيط وسريع للإضافة.
- لكن ليس الأفضل للبحث.

2. التخزين المتسلسل (Sequential File):

يُستخدم هذا النوع من تنظيم الملفات عندما تحتاج التطبيقات إلى معالجة جميع السجلات بشكل متسلسل (واحد وراء الآخر)، وليس الوصول العشوائي.

السجلات مرتبة بحسب Search-Key:

يعني أن البيانات داخل الملف مخزنة بترتيب تصاعدي أو تنازلي حسب قيمة مفتاح البحث (مثل رقم الهوية، الاسم، رقم الطالب..).

مثال:

إذا كان المفتاح هو Student_ID فسيتم ترتيب السجلات حسب رقم الطالب.

عمليات الحذف Deletion:

عند حذف سجل في هذا النوع من التنظيم، لا يتم حذف السجل فعلياً مباشرة.
بدل ذلك:

- يُستخدم Pointer Chain (سلسلة مؤشرات) لربط السجلات المحذوفة.
- هذا يسمح بمعرفة أماكن الفراغ لاحقاً عند الإدراج.

السجل يبقى موجوداً لكن يُربط بمؤشر يشير إلى أنه محذوف.

عمليات الإدراج insertion:

عند إدراج سجل جديد:

- تحديد المكان الصحيح حسب الترتيب ← هذا ضروري.
- لدينا حالتان:

1. إذا كان هناك مساحة فارغة بين السجلات:

يوضع السجل مباشرة في مكانه الصحيح.

2. إذا لم توجد مساحة كافية:

يُحفظ السجل الجديد في Overflow Block (كتلة إضافية خارج الملف الأساسي).

في كلتا الحالتين:

يجب تحديث المؤشرات Pointer Chain حتى يبقى التسلسل صحيحاً.



الحاجة إلى إعادة التنظيم Reorganization:

بمرور الوقت:

- تتراكم الكتل الفائضة Overflow Blocks.
- تزداد سلاسل المؤشرات مما يبطئ الوصول إلى البيانات.

لذلك:

يتم عمل Reorganization (إعادة تنظيم) دوري للملف لتحسين:

- ترتيب السجلات.
- دمج الكتل الفائضة.
- تنظيف السجلات المحذوفة.

A-217	Brighton	750	
A-101	Downtown	500	
A-110	Downtown	600	
A-215	Mianus	700	
A-102	Perryridge	400	
A-201	Perryridge	900	
A-218	Perryridge	700	
A-222	Redwood	700	
A-305	Round Hill	350	
A-888	North Town	800	

3. التجزئة (Hashing):

- نأخذ قيمة من السجل ونضعها داخل دالة تقطيع (hash).
- نتيجة الهاش تحدد أي كتلة نضع فيها السجل.
- ممتاز للبحث عن قيمة محددة بسرعة.
- لكنه غير مناسب لعمليات البحث بالنطاق.

4. تنظيم متعدد الجداول داخل ملف واحد (Multitable Clustering File Organization):

هو أسلوب تخزين يتم فيه وضع بيانات أكثر من جدول (Relation) داخل ملف واحد، بحيث تكون السجلات المرتبطة ببعضها موجودة معاً في نفس المكان (نفس ال Block).

يعني ندمج بيانات جداول متعددة داخل نفس الملف، بدل تخزين كل جدول في ملف منفصل.

لماذا نستخدم هذا النوع؟

الفكرة الأساسية:

إذا كان لديك جدولان بينهما علاقة قوية، مثل:

▪ Customer العميل.

▪ Depositor الحسابات البنكية الخاصة بهذا العميل.

فبدل أن يكون كل واحد منهما في ملف منفصل، يمكن وضعهما معاً بحيث يكون:

▪ سجل العميل Customer

<i>customer-name</i>	<i>customer-street</i>	<i>customer-city</i>
Hayes	Main	Brooklyn
Turner	Putnam	Stamford

▪ ومعه مباشرة سجلات الحسابات المرتبطة به depositor

<i>customer-name</i>	<i>account-number</i>
Hayes	A-102
Hayes	A-220
Hayes	A-503
Turner	A-305

في نفس ال block:

Hayes	Main	Brooklyn
Hayes	A-102	
Hayes	A-220	
Hayes	A-503	
Turner	Putnam	Stamford
Turner	A-305	

الألم يعلمك ما لا تعلمه الراحة

ويجعلك أنصحب مما كنت



الفائدة الرئيسية: تحسين الاستعلامات التي تربط الجداول ببعضها

مثال:

"أعطني بيانات العميل + جميع حساباته" او الاستعلامات من الشكل depositor ∞ customer
هذا الاستعلام سيكون أسرع لأن كل البيانات موجودة في مكان واحد وليس في ملفين مختلفين.
متى يكون سيئاً؟

عندما نريد استعلاماً يخص جدول واحد فقط مثل:

"أعطني جميع العملاء فقط"

هنا الأداء يصبح أبطأ لأن السجلات مختلطة مع سجلات depositor داخل نفس الملف.

فائدة اربيسيزية

كيف يتم حل مشكلة الاستعلام على جدول واحد في حال تخزين أكثر من جدول في ملف واحد؟
يمكن إضافة Pointer Chains سلاسل مؤشرات تربط سجلات الجدول نفسه ببعض.

مثال:

سلسلة تربط كل سجلات customer معاً فقط.

وسلسلة أخرى لسجلات depositor فقط.

Hayes	Main	Brooklyn	
Hayes	A-102		
Hayes	A-220		
Hayes	A-503		
Turner	Putnam	Stamford	
Turner	A-305		

مما يسهل الوصول للبيانات الخاصة بالجدول.

The End

النهايات ليست سقوطاً

بل بداية طريق لم نكن نراه